

FPT University

IN-DEPTH RESEARCH REPORT

Advanced Architectures in Conversational AI and Optimization Strategies for Semantic Parsing in Task-Oriented Dialogue Systems

Presented by: Group 3

Lecturer: Phan Thị Lệ Thuỳn

Members:

QE190039 Tô Thanh Hậu
QE190076 Ngô Tuấn Hoàng
QE190136 Đặng Văn Hậu
QE190155 Nguyễn Lê Tấn Pháp
QE190012 Nguyễn Châu Thành Sơn

Contents

Abstract	iii
List of Abbreviations	iv
1 Introduction	1
1.1 Semantic Parsing as the Core NLU Module	1
1.2 Why Model Evolution Matters for Text-to-SQL	1
2 Methodology and Scope	2
2.1 Data Source Constraint	2
2.2 Organization Strategy	2
3 Background: Semantic Parsing in TODS	2
3.1 Problem Definition	2
3.2 Reliability Requirements for Text-to-SQL	3
4 Literature Review: Model Evolution and Implications	3
4.1 Wave 1: Foundational Layer — BERT → T5	3
4.1.1 BERT (Encoder-only): strong understanding, limited generation	3
4.1.2 T5 (Encoder–Decoder): text-to-text mapping and SQL generation fit	3
4.2 Transformer Scaling: Dense Decoder-only Growth and Its Trade-offs	4
4.3 Wave 2: Optimization Race — Mistral and Falcon	4
4.3.1 Mistral: sliding window attention and cache optimization	4
4.3.2 Falcon: Multi-Query Attention for high-throughput serving	4
4.4 Instruction Tuning and Distillation — Vicuna and ChatGPT-style Assistants	4
4.4.1 Vicuna: knowledge distillation for ChatGPT-like behavior	4
4.4.2 ChatGPT (OpenAI): system-level shift for usefulness and iterative refinement	5
4.5 Sparse Scaling — Mixture-of-Experts (MoE)	5
4.6 Alignment (RLHF): usability gain vs correctness risk	5

4.7	Open-family Refinement — LLaMA 1–3 (and why it matters)	5
4.8	Reasoning Orientation — System 1 vs System 2	6
4.9	Wave 3: Agentic AI — Claude and long-horizon workflows	6
4.10	Multimodal Unification — Gemini	6
5	Comparative Analysis for Semantic Parsing	7
5.1	Architecture-level Comparison Table	7
5.2	System Recommendation: Pipeline View	7
6	Challenges and Future Directions	8
7	Conclusion	8
	References	8

Abstract

Modern chatbots are shifting from open-domain conversation to task-oriented dialogue systems (TODS), where the system must not only respond fluently but also complete actions reliably. In this setting, **semantic parsing** becomes a core capability: mapping natural language into structured logical forms such as SQL for safe execution in downstream systems. This report synthesizes architectural and training evolutions across major model families discussed in the provided research compilation: foundational encoder/encoder–decoder models (e.g., BERT, T5), decoder-only optimization-era LLMs (e.g., Mistral, Falcon, LLaMA/Vicuna), instruction-tuned assistants (ChatGPT-style), sparse scaling (MoE), alignment (RLHF), long-context and agentic workflows (Claude, reasoning-oriented models), and multimodal unification (Gemini). We compare paradigms using dimensions that directly affect Text-to-SQL robustness: representational capacity, inference efficiency, long-context handling, semantic validity, and agentic verification stability. The report concludes with system-level recommendations for deploying reliable semantic parsers using generate–check–repair loops and controlled test-time compute.

Keywords: Semantic parsing; Text-to-SQL; Task-oriented dialogue systems; LLMs; Transformer; Mixture-of-Experts; RLHF; Long-context; Agentic workflows; Multimodal.

List of Abbreviations

Abbreviation	Meaning
TODS	Task-Oriented Dialogue Systems
NLU	Natural Language Understanding
Semantic Parsing	Mapping natural language to executable logical forms (e.g., SQL)
Text-to-SQL	Generating SQL queries from natural language
LLM	Large Language Model
SFT	Supervised Fine-Tuning
RLHF	Reinforcement Learning from Human Feedback
MoE	Mixture-of-Experts
KV Cache	Key-Value Cache for Transformer inference
SWA	Sliding Window Attention

1. Introduction

Conversational AI is increasingly expected to complete goals (query databases, perform business actions, navigate workflows) rather than only maintain conversation. This shift elevates the importance of **Task-Oriented Dialogue Systems (TODS)** and introduces a high-stakes requirement: the system must transform user language into **executable actions** reliably.

1.1 Semantic Parsing as the Core NLU Module

In TODS, semantic parsing is a central component of the NLU layer, mapping a user utterance (and dialogue context) into a structured logical form such as SQL. Compared to intent classification alone, semantic parsing must preserve *semantic correctness* under schema constraints (tables, columns, joins, aggregation) and across multi-turn dialogue phenomena (ellipsis, coreference, evolving constraints).

1.2 Why Model Evolution Matters for Text-to-SQL

Text-to-SQL quality depends on:

- **Representation capacity:** capturing global dependencies, schema relations, compositional logic.
- **Inference efficiency:** cost/memory constraints in real deployment, especially long contexts.
- **Alignment and format compliance:** following instructions and producing structured outputs consistently.
- **Verification and correction:** detecting invalid SQL and repairing it before execution.

This report reviews model evolution through these deployment-driven lenses, building a logical chain from foundational models to optimized serving, instruction-tuned assistants (ChatGPT), and agentic systems (Claude/Gemini).

2. Methodology and Scope

2.1 Data Source Constraint

All technical content about model families and implications is derived from the provided compiled research document (the “research summary PDF”). The second PDF is used strictly as a **writing frame reference** (structure only), not as a factual data source.

2.2 Organization Strategy

We structure the literature review by evolutionary pressure:

1. **Foundational representation learning** (BERT/T5).
2. **Transformer scaling and serving constraints** (dense decoder-only growth).
3. **Optimization race** (Mistral/Falcon: memory + throughput).
4. **Instruction tuning and alignment** (ChatGPT-style behavior, RLHF).
5. **Sparse scaling** (MoE: capacity vs compute).
6. **Agentic reasoning and verification** (System-2, Claude).
7. **Multimodal unification** (Gemini).

3. Background: Semantic Parsing in TODS

3.1 Problem Definition

Semantic parsing in TODS can be summarized as:

- **Input:** user utterance + dialogue context.

- **Output:** executable logical form (SQL/API calls/DSL program).
- **Goal:** semantic validity and executability (not only fluent text).

3.2 Reliability Requirements for Text-to-SQL

A production-grade Text-to-SQL assistant must ensure:

1. **Schema grounding:** only valid tables/columns; correct join paths.
2. **Compositional correctness:** filters/grouping/order reflect user intent.
3. **Dialogue-state consistency:** constraints persist correctly across turns.
4. **Verification:** detect invalid SQL and repair before execution.

4. Literature Review: Model Evolution and Implications

4.1 Wave 1: Foundational Layer — BERT → T5

4.1.1 BERT (Encoder-only): strong understanding, limited generation

BERT uses bidirectional context to improve NLU. In semantic parsing systems, it is often effective for sub-tasks such as entity recognition, schema linking, or constraint extraction. However, encoder-only models are not natural end-to-end SQL generators.

4.1.2 T5 (Encoder–Decoder): text-to-text mapping and SQL generation fit

T5 reframes tasks as text-to-text transformations, which aligns well with NL→SQL mapping. It provides a structured pathway from understanding (encoder) to generation (decoder). The trade-off is that encoder–decoder inference can be heavier at scale compared to decoder-only serving.

4.2 Transformer Scaling: Dense Decoder-only Growth and Its Trade-offs

Transformers overcome pre-transformer seq²seq limitations (long dependencies, limited parallelization) via self-attention. Dense decoder-only scaling improves generation capability and generalization; however, it introduces deployment bottlenecks: every token activates all parameters and attention cost grows quickly with context length.

4.3 Wave 2: Optimization Race — Mistral and Falcon

4.3.1 Mistral: sliding window attention and cache optimization

Mistral emphasizes memory efficiency. Sliding Window Attention (SWA) reduces attention scope per token, helping control VRAM usage, while cache strategies reduce KV memory overhead. This makes the model more feasible in practical deployments where long contexts exist.

4.3.2 Falcon: Multi-Query Attention for high-throughput serving

Falcon targets inference bottlenecks in large-scale serving. By sharing Key/Value across heads (Multi-Query Attention), KV cache VRAM requirements can be reduced drastically, improving throughput. This matters for production assistants that serve many concurrent users.

4.4 Instruction Tuning and Distillation — Vicuna and ChatGPT-style Assistants

4.4.1 Vicuna: knowledge distillation for ChatGPT-like behavior

Vicuna illustrates that instruction-following style can be transferred through supervised fine-tuning on dialogue data (distillation from stronger teachers). This improves interactive quality and instruction compliance.

4.4.2 ChatGPT (OpenAI): system-level shift for usefulness and iterative refinement

ChatGPT represents a practical milestone: it emphasizes instruction-following, interactive usefulness, and multi-turn refinement—traits that strongly matter for semantic parsing in TODS. In Text-to-SQL workflows, ChatGPT-style assistants are valuable because they can:

- ask clarification questions when the user intent is ambiguous,
- follow strict output formats (SQL template, JSON wrappers),
- iteratively refine queries across multiple turns.

However, fluent responses do not guarantee *semantic validity*. Therefore, ChatGPT-style generation must be coupled with deterministic grounding and verification.

4.5 Sparse Scaling — Mixture-of-Experts (MoE)

Dense models are expensive because all parameters run per token. **MoE** scales capacity by routing tokens to a small number of experts (top- k), increasing total parameters without proportional per-token compute. This is a major direction for combining high capability with deployable cost.

4.6 Alignment (RLHF): usability gain vs correctness risk

RLHF improves instruction following and safety, but it can push models to produce outputs that are helpful-sounding even when uncertain. For semantic parsing, this creates a serious risk: confident but invalid SQL. Therefore, alignment should be treated as a usability layer, while correctness must be enforced via external checks.

4.7 Open-family Refinement — LLaMA 1–3 (and why it matters)

The LLaMA family illustrates iterative improvement:

- stronger general capability at fixed compute budgets,
- better instruction following and prompt retention,
- but complex tool-use stability often still requires agentic scaffolding.

In semantic parsing, open models are effective building blocks for modular pipelines (draft generation + external verification).

4.8 Reasoning Orientation — System 1 vs System 2

A deployment-critical distinction:

- **System 1:** fast, direct generation (good for simple queries).
- **System 2:** deliberate reasoning with self-correction (better for complex SQL).

Complex Text-to-SQL benefits from multi-step workflows that can revisit joins/filters and repair errors before returning final SQL.

4.9 Wave 3: Agentic AI — Claude and long-horizon workflows

Agentic systems focus on planning and iterative correction. In practice, they fit semantic parsing well because reliable systems often require:

1. draft SQL,
2. validate schema constraints,
3. simulate/execute,
4. repair and re-run until valid.

Long-context support further enables grounding against long documents/specifications, improving correctness in realistic deployments.

4.10 Multimodal Unification — Gemini

Multimodal-by-design approaches learn unified representations across text, images, audio, and code. This matters because real TODS increasingly rely on multimodal evidence (UI screenshots, PDFs, forms). A unified model can support “semantic parsing beyond text,” such as extracting fields from an interface and generating structured actions.

5. Comparative Analysis for Semantic Parsing

5.1 Architecture-level Comparison Table

Paradigm	Strengths for Text-to-SQL	Main Risks / Limits
Foundational (BERT/T5)	Strong NLU grounding; seq2seq mapping aligns with NL→SQL	Encoder-only not generative; encoder-decoder may be expensive at scale
Optimization race (Mistral/Falcon)	Feasible serving (memory, throughput); better deployment	Efficiency tricks may lose global reasoning; needs verification
Instruction-tuned (ChatGPT/Vicuna style)	Excellent instruction following; multi-turn refinement	Fluency ≠ validity; can hallucinate SQL without constraints
Sparse scaling (MoE)	High capacity at controlled compute cost	Still needs system checks for structured correctness
Agentic (Claude / System-2)	Generate-check-repair loops; tool-use stability; long-horizon	Performance depends on scaffolding (steps, context, compute)
Multimodal (Gemini)	Grounding with UI/docs; supports real-world TODS evidence	Multimodal errors can propagate; evaluation is harder

Table 5.1: Model paradigms compared for semantic parsing robustness in TODS.

5.2 System Recommendation: Pipeline View

A reliable semantic parsing assistant should be designed as a system, not only a model:

1. **Understanding module:** extract constraints + schema linking (NLU strength).
2. **Generation module:** draft SQL with strict formatting.
3. **Verification module:** schema/type checks; optional execution feedback.
4. **Repair loop:** revise SQL iteratively (agentic workflow).
5. **Alignment layer:** ensure safe and consistent interaction (separate from correctness).

6. Challenges and Future Directions

- **Semantic validity vs fluent hallucination:** RLHF improves style, not guaranteed correctness.
- **Long-context reliability:** bigger context helps but still needs retrieval/chunking + checks.
- **Agent-native semantic parsing:** future models should plan, validate, and repair natively.
- **Multimodal semantic parsing:** next TODS will parse from UI/doc evidence, not only text.

7. Conclusion

Model evolution from BERT/T5 foundations to optimized decoder-only LLMs (Mistral/Falcon), instruction-tuned assistants (ChatGPT-style), sparse scaling (MoE), agentic long-horizon workflows (Claude/System-2), and multimodal unification (Gemini) shows a consistent trend: **capability must be paired with verification**. For Text-to-SQL in TODS, the best results come from a system approach that combines strong generation with explicit grounding constraints and iterative repair loops before execution.

References

1. *Nghiên cứu Chatbots và Semantic Parsing.pdf* (primary compiled research source used for all model summaries and claims).
2. *A Comprehensive Survey on Comparisons Across Contextual...pdf* (used only as a writing-structure/frame reference, not as factual model data).